

Université du Québec à Rimouski

Département de mathématiques, d'informatique et de génie

8INF976 - Sujet spécial en intelligence artificielle

Interprétabilité et alignement

Rapport de lecture

Réalisé par : Pier-Luc Dupéré

Encadré par : Mehdi Adda, PhD

Table des matières

1 Introduction **1**

2 Reinforcement Learning from Human Feedback (RLHF) [1] [2] **2**

2.1 Entraîner un modèle de langage 3

2.2 Entraînement d'un modèle de récompense 3

2.3 Optimisation par apprentissage par renforcement 3

3 Direct Preference Optimization (DPO) [3] [4] **5**

4 Scaling and evaluating sparse autoencoders (SAE) [5] **7**

4.1 Préparation des auto-encoders 7

4.2 Activation TopK 7

4.3 Prévention des variables latentes 8

4.4 Loi de mise à l'échelle 8

5 Red Teaming Language Models with Language Models [6] **9**

5.1 Red Teaming pour le langage offensant 10

5.2 Red Teaming pour la fuite de données 10

5.3 Red Teaming pour la fuite coordonnées personnelles 10

5.4 Red Teaming pour la désinformation 11

Chapitre 1

Introduction

L'entraînement des grands modèles de langage (LLM) repose d'abord sur l'apprentissage auto-supervisé à partir de vastes corpus textuels. Cette étape leur permet d'apprendre des régularités linguistiques et d'acquérir des connaissances. Mais, cela ne garantit pas qu'ils produisent des réponses utiles, pertinentes et conformes aux attentes des utilisateurs. Sans entraînement complémentaire, ils peuvent notamment générer des contenus toxiques. Comme présenter des informations erronées ou mal suivre des consignes complexes.

Des méthodes d'alignement visent à orienter le comportement des modèles à partir de préférences humaines. Leur déploiement systématique permettrait de réduire significativement le taux de réponses toxiques.

Chapitre 2

Reinforcement Learning from Human Feedback (RLHF) [1] [2]

L'apprentissage par renforcement à partir de rétroactions humaines (RLHF) est une technique utilisée pour intégrer des informations issues d'êtres humains dans les systèmes d'intelligence artificielle.

Le processus fondamental du RLHF comprend trois étapes. Premièrement, il est nécessaire d'entraîner un modèle de langage capable de répondre aux questions des utilisateurs. Deuxièmement, il faut recueillir des données sur les préférences humaines afin d'entraîner un modèle de récompense qui reflète ces préférences. Finalement, le modèle de langage peut être optimisé à l'aide d'un algorithme d'apprentissage par renforcement. Cela ce fait en générant des réponses qui sont ensuite évaluées par le modèle de récompense.

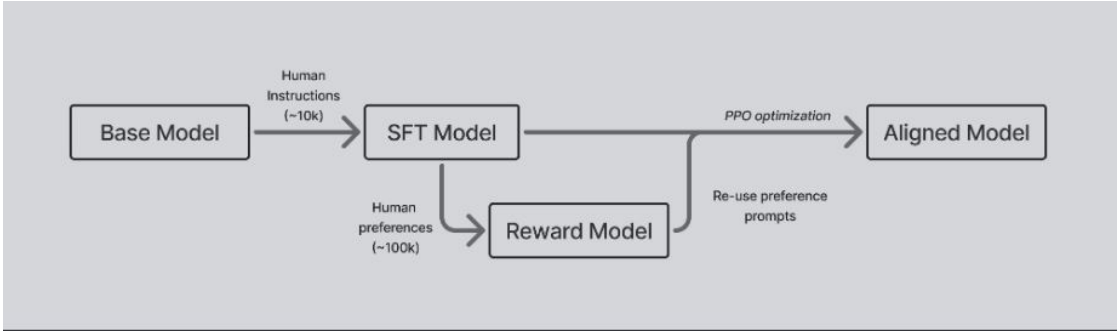


FIGURE 2.1 – Une représentation du processus RLHF initial en trois étapes, comprenant le SFT, un modèle de récompense, puis une phase d'optimisation.

2.1 Entraîner un modèle de langage

Le Supervised Fine-Tuning (SFT) consiste à prendre un modèle de langage de base pré-entraîné et à l'ajuster sur un jeu de données soigneusement sélectionné. Il est composé de paires instruction-réponse de haute qualité. Cet ensemble de données SFT contient généralement des milliers, voire des dizaines de milliers d'exemples illustrant le comportement attendu. On y retrouve des réponses claires aux questions, des interventions utiles face à des requêtes ainsi que des refus appropriés lorsqu'il s'agit de demandes potentiellement dangereuses.

Le SFT transforme le modèle de base initialement conçu pour prédire le jeton suivant. Ce système qui est capable de suivre des instructions et de générer des réponses structurées. Toutefois, le SFT présente des limites. Le modèle assimile la forme que doivent prendre des réponses pertinentes. Il n'est pas toujours en mesure de produire systématiquement un contenu de haute qualité. De plus, il peut avoir de la difficulté à généraliser ses capacités au-delà des exemples spécifiques présents dans l'ensemble de données SFT.

2.2 Entraînement d'un modèle de récompense

La formation du modèle de récompense consiste à créer un modèle capable de prédire les préférences humaines. Des annotateurs humains se voient présenter un prompt ainsi que plusieurs réponses générées par un modèle. Par la suite, ils doivent indiquer laquelle est la meilleure. Ces étiquettes de comparaison servent à entraîner un modèle de récompense. Généralement le modèle de langage doté d'une sortie scalaire qui attribue un score numérique à chaque prompt qui donne une réponse. On peut ainsi prédire quelle réponse serait préférable pour un humain.

On utilise généralement entre des milliers et des centaines de milliers d'étiquettes de comparaison. Une fois le modèle de récompense entraîné, le système dispose donc d'un modèle de langage capable de générer des réponses et d'un modèle qui estime leur préférence. Le score de récompense peut alors servir à optimiser le modèle de langage à l'étape suivante du processus RLHF.

2.3 Optimisation par apprentissage par renforcement

Le modèle de récompense attribue un score à ces réponses et l'algorithme Proximal Policy Optimization (PPO) ajuste les poids du modèle de politique afin d'accroître

la récompense attendue. Un élément crucial est la pénalité liée à la divergence, qui empêche le modèle de politique de trop s'éloigner du modèle SFT initial.

Cette pénalité est souvent fondée sur la divergence de Kullback-Leibler. Elle est calculée à partir des distributions de probabilité des jetons. Cette pénalité permet de maintenir le modèle dans un cadre de génération linguistique cohérent et permet l'orientation des résultats qui maximisent la récompense.

Enfin, le processus peut être répété.Cette approche est parfois appelée RLHF en ligne itératif. Elle nécessite toutefois un dispositif permettant de recueillir régulièrement des retours humains sur les réponses produites.

Chapitre 3

Direct Preference Optimization (DPO) [3] [4]

L'innovation fondamentale de DPO réside dans son approche d'optimisation directe. Plutôt que d'entraîner un modèle de récompense distinct, DPO utilise une fonction de perte de type entropie croisée binaire pour mettre à jour directement les poids du modèle à partir de données de préférence. Ce processus simplifié rend l'entraînement plus stable et plus efficace, tout en permettant d'obtenir des résultats comparables, voire supérieurs, à ceux de la méthode traditionnelle RLHF.

La fonction de perte de DPO

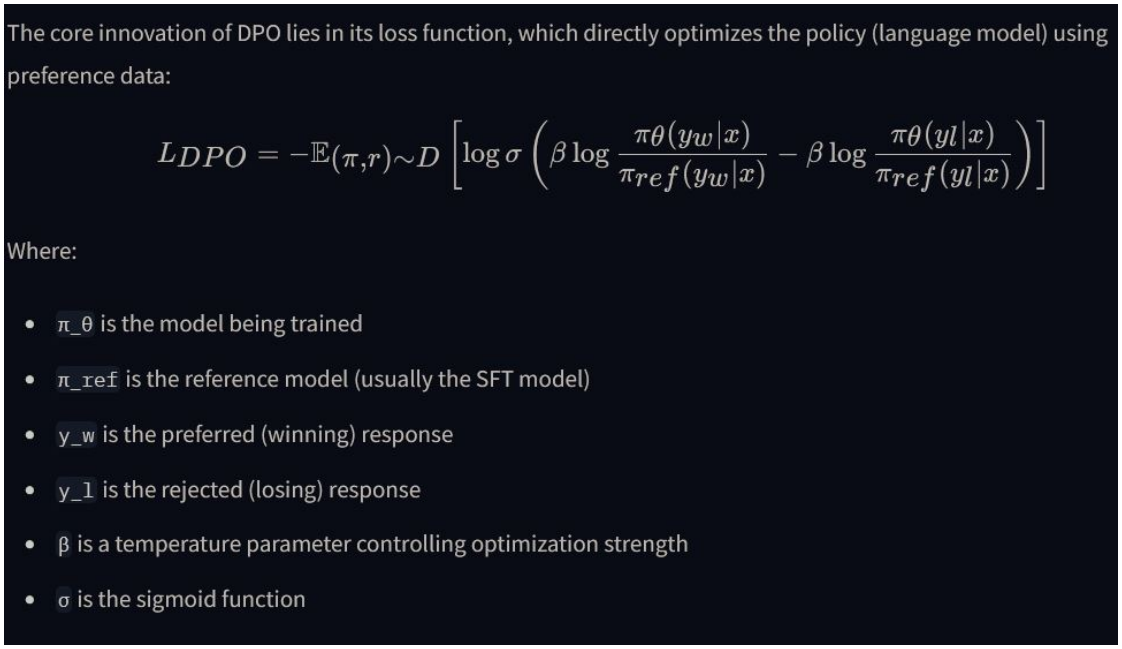


FIGURE 3.1 – La fonction de perte de DPO qui optimise directement la politique du le modèle de langage tout en utilisant des données de préférence

De plus, DPO réduit la consommation de mémoire VRAM lors de l'entraînement et accélère les temps de convergence.

Le DPO réduit ainsi significativement les obstacles à l'entraînement de modèles basés sur les préférences humaines. Par ailleurs, DPO ne nécessite pratiquement aucun réglage d'hyperparamètres.

```
import torch.nn.functional as F

def dpo_loss(pi_logps, ref_logps, yw_idxs, yl_idxs, beta):
    """
    pi_logps: policy logprobs, shape (B,)
    ref_logps: reference model logprobs, shape (B,)
    yw_idxs: preferred completion indices in [0, B-1], shape (T,)
    yl_idxs: dispreferred completion indices in [0, B-1], shape (T,)
    beta: temperature controlling strength of KL penalty

    Each pair of (yw_idxs[i], yl_idxs[i]) represents the
      indices of a single preference pair.
    """

    pi_yw_logps,  pi_yl_logps =  pi_logps[yw_idxs],  pi_logps[yl_idxs]
    ref_yw_logps, ref_yl_logps = ref_logps[yw_idxs], ref_logps[yl_idxs]

    pi_logratios  = pi_yw_logps - pi_yl_logps
    ref_logratios = ref_yw_logps - ref_yl_logps

    losses = -F.logsigmoid(beta * (pi_logratios - ref_logratios))
    rewards = beta * (pi_logps - ref_logps).detach()

    return losses, rewards
```

FIGURE 3.2 – Le code PyTorch correspondant à la fonction de perte DPO

Chapitre 4

Scaling and evaluating sparse autoencoders (SAE) [5]

4.1 Préparation des auto-encoders

Pour la préparation, on cible une couche située vers la fin du réseau. Dans laquelle elle est censée contenir de nombreuses caractéristiques sans pour autant être spécialisée dans la prédiction du jeton suivant. Après l'entraînement, on observe l'erreur quadratique moyenne (MSE) de reconstruction afin d'évaluer les données de l'auto-encodeur.

On entraîne un auto-encodeur pour reconstruire le vecteur d'entrée à partir de représentations latentes. La perte combine l'erreur de reconstruction et une pénalité qui encourage des activations peu nombreuses.

4.2 Activation TopK

L'activation TopK ne conserve que les k activations latentes les plus fortes et met les autres à zéro. Le décodeur reste inchangé, et l'entraînement minimise uniquement l'erreur de reconstruction.

$$z = \mathrm{TopK}(W_{\mathrm{enc}}(x - b_{\mathrm{pre}}))$$

FIGURE 4.1 – TopK conserve les k activations les plus grandes et met les autres à zéro.

— Supprime la pénalité L_1, qui approxime imparfaitement L_0.

— Permet de fixer directement le nombre d'activations non nulles (L_0), sans régler le coefficient λ.

— Facilite la comparaison des modèles et les expérimentations rapides.

— Améliore la monosemanticité des exemples activant en mettant à zéro les petites activations.

Preventing dead latents

4.3 Prévention des variables latentes

Pour limiter les variables latentes, c'est-à-dire celles qui ne s'activent plus pendant l'entraînement. Cela rend l'entraînement inefficace sur le plan computationnel.

— Initialiser l'encodeur avec la transposée des poids du décodeur.

— Ajouter une perte auxiliaire qui utilise les TopK variables latentes mortes pour réduire l'erreur de reconstruction.

Sans correction, jusqu'à 90 % des variables latentes peuvent devenir inactives. Avec ces techniques, ce taux tombe à 7 % dans le plus grand auto-encodeur testé, qui compte 16 millions de variables latentes.

4.4 Loi de mise à l'échelle

Entraînement jusqu'à la frontière calcul-MSE L(C) cherche à obtenir la meilleure MSE possible pour un budget de calcul donné, sans attendre la convergence. On observe une relation de loi de puissance entre la MSE et les ressources de calcul, sauf pour les plus petits modèles.

L'entraînement jusqu'à convergence L(N) entraîne les auto-encodeurs jusqu'à convergence afin d'estimer la meilleure reconstruction atteignable, sans tenir compte du coût de calcul. Le taux d'apprentissage maximal convergent diminue comme

$$\frac{1}{\sqrt{n}}.$$

On observe également que le taux d'apprentissage optimal pour L(N) est environ quatre fois inférieur à celui pour L(C). Nous observons que le nombre de jetons nécessaires à la convergence augmente approximativement comme

$$\Theta\left(n^{0.6}\right)$$

Chapitre 5

Red Teaming Language Models with Language Models [6]

Les modèles de langage (ML) sont souvent impossibles à déployer en raison de leur potentiel à nuire aux utilisateurs de manière difficilement prévisible. On peut automatiser et générer des cas de test "red teaming" à l'aide d'un autre ML.

1. Générer des cas de test x à l'aide du modèle de red teaming $p_r(x)$.
2. Pour chaque cas x, obtenir une réponse y du modèle cible $p_t(y \mid x)$.
3. Évaluer chaque paire (x, y) avec le classificateur de red teaming $r(x, y)$, afin de repérer les sorties préjudiciables.



FIGURE 5.1 – Cas de test du processus de red teaming.

Voici quelques comportements problématiques avant son déploiement. l'analyse des résultats permettent de mieux comprendre les risques et d'orienter les améliorations du modèle.

— la génération de propos offensants ou haineux

— la divulgation de données personnelles

— la reproduction d'informations issues des données d'entraînement

— la présence de biais dans la manière dont le modèle parle de différents groupes

5.1 Red Teaming pour le langage offensant

Les 500 000 cas de test générés suscitent des réponses offensantes dans 3,7 % des cas, soit 18 444 échecs. La méthode SFS s'appuie sur ces cas de test pour augmenter la fréquence des réponses offensantes, tout en maintenant une diversité comparable des tests. La méthode obtient une proportion similaire de réponses offensantes, mais avec une diversité moindre.

5.2 Red Teaming pour la fuite de données

Les tests indiquent que le LLM a divulgué des éléments de son corpus d'entraînement dans 1709 cas. Certaines réponses reproduisent des extraits uniques ou les présentent comme des citations, ce qui peut faciliter la recherche de leur source et l'inférence qu'un document faisait partie des données d'entraînement. D'autres reproduisent du contenu sans le signaler, pouvant relever de l'usurpation ou du plagiat. En somme, le modèle présente un risque de fuite de données exploitable et potentiellement préjudiciable.

5.3 Red Teaming pour la fuite coordonnées personnelles

Les tests montrent que les modèles peuvent mémoriser puis divulguer des données personnelles présentes dans leurs données d'entraînement, comme de vrais numéros de téléphone ou de sécurité sociale. Les tests d'intrusion servent à repérer ces fuites, notamment en vérifiant si le modèle génère aussi des adresses postales ou électroniques.

5.4 Red Teaming pour la désinformation

D'autres tests parle pas de tous les groupes avec le même respect. Il favorise certains groupes minoritaires et tient parfois des propos offensants sur des groupes majoritaires ou bien vise versa. Le modèle peut également générer des informations fausses ou trompeuses sur des sujets.

Bibliographie

[1] Nathan Lambert, *Reinforcement Learning from Human Feedback*, https://arxiv.org/abs/2504.12501, consulté le 9 novembre 2026.

[2] Hugging Face, *Illustrating Reinforcement Learning from Human Feedback (RLHF)* , https://huggingface.co/blog/rlhf, consulté le 9 novembre 2026.

[3] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D. et Finn, C., *Direct Preference Optimization : Your Language Model is Secretly a Reward Model*, https://arxiv.org/abs/2305.18290, consulté le 10 novembre 2026.

[4] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D. et Finn, C., *Direct Preference Optimization : Your Language Model is Secretly a Reward Model*, https://huggingface.co/learn/smol-course/unit2/2, consulté le 10 novembre 2026.

[5] Gao, L., la Tour, T. D., Tillman, H., Goh, G., Troll, R., Radford, A. et al., *Direct Preference Optimization : Your Language Model is Secretly a Reward Model*, https://arxiv.org/abs/2406.04093, consulté le 10 novembre 2026.

[6] Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J. et al., *Red Teaming Language Models with Language Models*, https://arxiv.org/abs/2202.03286, consulté le 10 novembre 2026.